Database: NoSQL (MongoDB)

Section B — Practical Coding Tasks

Task 1: Set Up the QuickEats Restaurant Database

Create database and collection:

use quickeats_db

db.createCollection("restaurants")

Insert 5 restaurants using insertMany:

```
db.restaurants.insertMany([
    {
        name: "Spice Garden",
        cuisine: "Indian",
        city: "Ahmedabad",
        rating: 4.6,
        isOpen: true,
        menu: [
            { itemName: "Paneer Tikka", price: 280 },
            { itemName: "Butter Naan", price: 60 },
            { itemName: "Biryani", price: 320 }
        ]
    },
    {
        name: "Pizza Paradise",
        cuisine: "Italian",
        city: "Surat",
        rating: 4.8,
        isOpen: true,
        menu: [
            { itemName: "Margherita Pizza", price: 350 },
            { itemName: "Pasta Alfredo", price: 300 },
            { itemName: "Garlic Bread", price: 150 }
        ]
    },
    {
        name: "Dragon Palace",
        cuisine: "Chinese",
        city: "Vadodara",
        rating: 4.4,
        isOpen: true,
        menu: [
            { itemName: "Hakka Noodles", price: 220 },
            { itemName: "Manchurian", price: 240 },
```

```
            { itemName: "Fried Rice", price: 200 }
        ]
    },
    {
        name: "Burger Hub",
        cuisine: "American",
        city: "Ahmedabad",
        rating: 4.3,
        isOpen: false,
        menu: [
            { itemName: "Classic Burger", price: 220 },
            { itemName: "French Fries", price: 120 },
            { itemName: "Chocolate Shake", price: 180 }
        ]
    },
    {
        name: "Sushi World",
        cuisine: "Japanese",
        city: "Surat",
        rating: 4.7,
        isOpen: true,
        menu: [
            { itemName: "Veg Sushi", price: 350 },
            { itemName: "Ramen", price: 300 },
            { itemName: "Tempura", price: 280 }
        ]
    }
])
```

Display all restaurants:

db.restaurants.find()

For a cleaner screenshot:

db.restaurants.find().pretty()

Task 2: Query and Filter Orders Using Operators

Create the orders collection:

db.createCollection("orders")

Insert 8 orders:

```
db.orders.insertMany([
    {
        customerId: "C101",
        restaurantName: "Spice Garden",
```

```
        items: [
            { itemName: "Paneer Tikka", quantity: 2, price: 280 },
            { itemName: "Biryani", quantity: 1, price: 320 }
        ],
        totalAmount: 880,
        status: "delivered",
        orderDate: ISODate("2026-08-08T12:30:00Z")
    },
    {
        customerId: "C102",
        restaurantName: "Pizza Paradise",
        items: [
            { itemName: "Margherita Pizza", quantity: 1, price: 350 },
            { itemName: "Garlic Bread", quantity: 1, price: 150 }
        ],
        totalAmount: 500,
        status: "confirmed",
        orderDate: ISODate("2026-08-09T14:00:00Z")
    },
    {
        customerId: "C103",
        restaurantName: "Dragon Palace",
        items: [
            { itemName: "Hakka Noodles", quantity: 2, price: 220 },
            { itemName: "Manchurian", quantity: 1, price: 240 }
        ],
        totalAmount: 680,
        status: "delivered",
        orderDate: ISODate("2026-08-10T18:30:00Z")
    },
    {
        customerId: "C104",
        restaurantName: "Burger Hub",
        items: [
            { itemName: "Classic Burger", quantity: 2, price: 220 },
            { itemName: "French Fries", quantity: 1, price: 120 }
        ],
        totalAmount: 560,
        status: "pending",
        orderDate: ISODate("2026-08-11T13:00:00Z")
    },
    {
        customerId: "C105",
        restaurantName: "Sushi World",
        items: [
            { itemName: "Veg Sushi", quantity: 2, price: 350 },
            { itemName: "Ramen", quantity: 1, price: 300 }
        ],
```

```
        totalAmount: 1000,
        status: "delivered",
        orderDate: ISODate("2026-08-12T19:00:00Z")
    },
    {
        customerId: "C106",
        restaurantName: "Spice Garden",
        items: [
            { itemName: "Biryani", quantity: 2, price: 320 }
        ],
        totalAmount: 640,
        status: "confirmed",
        orderDate: ISODate("2026-08-13T12:00:00Z")
    },
    {
        customerId: "C107",
        restaurantName: "Pizza Paradise",
        items: [
            { itemName: "Pizza", quantity: 2, price: 350 }
        ],
        totalAmount: 700,
        status: "delivered",
        orderDate: ISODate("2026-08-14T20:00:00Z")
    },
    {
        customerId: "C108",
        restaurantName: "Dragon Palace",
        items: [
            { itemName: "Fried Rice", quantity: 1, price: 200 },
            { itemName: "Manchurian", quantity: 1, price: 240 }
        ],
        totalAmount: 440,
        status: "pending",
        orderDate: ISODate("2026-08-15T11:30:00Z")
    }
])
```

Query 1: $gt and $eq

Find orders where:

- totalAmount is greater than Rs 400
- status is exactly "delivered"

```
db.orders.find({
    totalAmount: { $gt: 400 },
    status: { $eq: "delivered" }
})
```

A shorter equivalent is:

```
db.orders.find({
    totalAmount: { $gt: 400 },
    status: "delivered"
})
```

Query 2: $in, projection, sorting, and limiting

Find orders whose status is either pending or confirmed, display only the required fields, sort by amount descending, and return at most 5:

```
db.orders.find(
    {
        status: { $in: ["pending", "confirmed"] }
    },
    {
        _id: 0,
        customerId: 1,
        totalAmount: 1,
        status: 1
    }
).sort({
    totalAmount: -1
}).limit(5)
```

Task 3: Update Order Records Using Update Operators

3.1 Change one pending order to confirmed

First identify a pending order:

db.orders.find({ status: "pending" }).pretty()

Then update one specific order. For example, using customer C104:

```
db.orders.updateOne(
    {
        customerId: "C104",
        status: "pending"
    },
    {
        $set: {
            status: "confirmed",
            confirmedAt: new Date()
        }
    }
)
```

Verify:

```
db.orders.find({
    customerId: "C104"
}).pretty()
```

3.2 Mark all confirmed orders as delivered

```
db.orders.updateMany(
    {
        status: "confirmed"
    },
    {
        $set: {
            status: "delivered"
        }
    }
)
```

Verify:

```
db.orders.find({
    status: "delivered"
}).pretty()
```

3.3 Add a delivery-log entry using $push

Add the dispatched event to the order belonging to C104:

```
db.orders.updateOne(
    {
        customerId: "C104"
    },
    {
        $push: {
            deliveryLog: {
                event: "dispatched",
                time: new Date()
            }
        }
    }
)
```

Verify:

```
db.orders.find({
    customerId: "C104"
}).pretty()
```

3.4 Remove confirmedAt using $unset

```
db.orders.updateOne(
    {
        customerId: "C104"
    },
    {
        $unset: {
            confirmedAt: ""
        }
    }
)
```

Verify:

```
db.orders.find({
    customerId: "C104"
}).pretty()
```

Task 4: Restaurant Revenue Report Using Aggregation

The pipeline below:

1. $match → selects only delivered orders.
2. $group → calculates total revenue and number of orders per restaurant.
3. $project → displays only the requested fields.
4. $sort → ranks restaurants by revenue.
5. $limit → returns the top 3.

```
  db.orders.aggregate([
    {
        $match: {
            status: "delivered"
        }
    },
    {
        $group: {
            _id: "$restaurantName",
            totalRevenue: {
                $sum: "$totalAmount"
            },
            orderCount: {
                $sum: 1
            }
        }
    },
    {
```

```
        $project: {
            _id: 0,
            restaurantName: "$_id",
            totalRevenue: 1,
            orderCount: 1
        }
    },
    {
        $sort: {
            totalRevenue: -1
        }
    },
    {
        $limit: 3
    }
])
```